No-arbitrage for lift lending

Bounded TAO-cycles, μ's F, and the lift/readd inverse

Subtensor lending (PR #3268)
Working note — increment 3c (fixed-weight vs Uni v2 + Jacob rec)

2026-10-29

Abstract

Following μ's method: for each non-trivial action cycle that **starts and ends holding only TAO**, write the terminal TAO as a factor of the initial holding, sized as a fraction h of pool TAO, as a function of LP state (y, x, w_1) and lending parameters. Then minimise / maximise that factor over the parameter space and ask whether any region has expected value > 1 (attacker profit).

Increments 1–2 recorded μ's F, Cycle A, and the failure of EMA/capacity protections under weight-updating re-add. **Increment 3c** records the clone drain; answers μ's "abandon Balancer / just Uniswap v2?" (no: swaps were already fixed-weight weighted CPMM; Uni v2 is only $w = \frac{1}{2}$; only lending readd skips asymmetric-join reweight); compares designs (a)–(e); reports a bounded cycle search; and states a recommendation for Jacob (ship (a); harden with (b) if needed). Full closed-form proof remains open.

Contents

1 Method (from μ) **1**

2 State and parameters **2**
2.1 LP / Balancer state . . . 2
2.2 Lending book (per subnet) . . . 2
2.3 Lending parameters Θ (defaults) . . . 2

3 μ's September F (earlier proposal) **2**
3.1 Provisional variable map into current lift math . . . 3

4 Does the same cycle exist today? **3**

5 TAO-pure cycles under the current design **3**
5.1 Cycle A — long wallet ratchet (primary drain candidate) . . . 3
5.2 Cycle B — short wallet mirror . . . 4
5.3 Cycle C — raise then wallet-close . . . 4
5.4 Cycle D — escrow closes only . . . 4
5.5 Cycle E — partial repay interleaved with swaps . . . 4

6 Where a fee-only proof fails **4**

7 Numerics: SN29-like and μ's F **5**
7.1 μ's F vs size 5
7.2 Cycle A with weight update (counterexample) 5

8 Theorem status (increment 1) **6**

9 h-sweep and μ's returns graph **6**
9.1 μ's graph (September), eyeball check 6
9.2 Cycle A under current lift (production fee) 6

10 Do existing protections keep $EV \leq 0$? **7**

11 Clone reproduction (spec 493) **7**

12 Answer to mu: we do not abandon Balancer **8**

13 Two readd designs **8**

14 Root fix = design (a) **9**

15 Bounded cycle search **9**

16 Follow-up: "is this just Uniswap v2?" **10**
16.1 What main actually re-derives 10
16.2 Fixed weights $\neq$ Uniswap v2 10
16.3 What is true while a lift is outstanding 10
16.4 What is lost versus "full" Balancer join behaviour? 10

17 Designs that keep re-derivation elsewhere **11**

18 Recommendation for Jacob **11**

19 Next increments **12**

1 Method (from μ)

1. Enumerate every non-trivial finite sequence of stake/unstake swaps and lending calls that **begins and ends with a pure-TAO book** (no open position, no residual alpha inventory attributed to the cycle).

2. Write terminal TAO V' as
$$V' = F_{\text{cyc}}(h;\, y, x, w_1;\, \Theta)\, V,$$
where V is the initial TAO stake of the cycle, $h = V/y$ (or another natural scale against pool TAO y), and Θ collects lending parameters (LTV, thin LTV, swap fee, EMA band, APR, term, capacity floors).

3. Over a compact parameter domain (including SN29-like and deep equal-weight pools), compute $\min F_{\text{cyc}}$ and $\max F_{\text{cyc}}$.

4. Positive EV for that cycle $\iff$ some feasible point has $F_{\text{cyc}} > 1$ after fees (attacker convention below).

Remark 1 (Sign convention). We take $F_{\text{cyc}} > 1$ as **attacker profit** (ends with more TAO than started, marking-to-market any returned alpha at the exit swap). μ's September note used "profitable when $F < 1$": that F is a **cost / survival factor for the pool (or for the round-trip price)**, i.e. the reciprocal convention. We keep μ's symbol F for the September formula unchanged, and write F_{cyc} for attacker-return factors. Relation when both apply to the same cycle: $F_{\text{cyc}} = 1/F$ up to fee placement.

2 State and parameters

2.1 LP / Balancer state

- Reserves: TAO y, alpha x.
- Weights: w_1 (base / alpha), $w_2 = 1 - w_1$ (quote / TAO). Spot $p = (w_1/w_2)\,(y/x)$.
- Invariant (Balancer): $L = x^{w_1} y^{w_2}$. At $w_1 = w_2 = \frac{1}{2}$, $L = \sqrt{xy}$ ("$\sqrt{k}$").
- Swap fee: protocol default $\phi_{\text{swap}} = 33/65535 \approx 5.03 \cdot 10^{-4}$. In μ's formula the fee symbol is φ; see Section 3.1.

2.2 Lending book (per subnet)

Lift book $(L_S, L_L, E_S, B_L, \ldots)$, positions keyed by (coldkey, netuid), EMA reference for the open band, G_{ema} for capacity. One-year term and 100% APR burn-all interest are part of Θ but are **idle** inside a same-block or few-block cycle (interest ≈ 0).

2.3 Lending parameters Θ (defaults)

Symbol	Default
λ_S (short LTV)	0.25
λ_L (long LTV)	0.25 (0.15 thin)
ϕ_{swap}	33/65535
β (EMA band)	0.15
floor / thin floor / ceiling	0.33 / 0.15 / 0.50
side split	50/50
APR / term	100% / 1 year

3 μ's September F (earlier proposal)

Transcription checked against the hand-derived image:

$$F = (f_h + 1)\left(\frac{(1-\varphi)^{1/w_2}}{(1-\varphi)^{1/w_2} + f_h - \varphi}\right)^{E}, \tag{1}$$

with exponent

$$E = \frac{f_h + 1}{(f_h + 1 + \varphi)\left(1 - \dfrac{1 - (1-\varphi)^{1/w_2}}{f_h - \varphi + 1}\right)^{w_2/(1-w_2)}}. \tag{2}$$

Conjecture 1 (μ, earlier design)**.** *Along the September shorting cycle, the pool (or round-trip) factor is F above, and the cycle is attacker-profitable precisely when $F < 1$.*

3.1 Provisional variable map into current lift math

The September proposal predated forced-escrow longs and pull-on-borrow lifts. A working identification used in this note (to be stress-tested against μ's full derivation when available):

Symbol	September reading	Current lift reading
w_2	quote weight	Balancer quote weight w_q (TAO leg)
φ	swap fee taken on the trade	ϕ_{swap} (same order)
f_h	haircut / size as fraction of pool TAO	candidate: $h = P/y$ after lift, or LTV-scaled $h = \lambda \cdot C/y$

Remark 2. If f_h is the **principal as a fraction of post-lift (or pre-trade) pool TAO**, then for a capacity-bound open $h \lesssim 0.075$–0.165 on a side (thin vs floor $\times$ split). For LTV-bound opens with collateral $C = \alpha y$, $P = \lambda C$ gives $f_h = \lambda\alpha$.

4 Does the same cycle exist today?

Cycle 1 (September-style short round trip)**.** Open a forced short of principal P alpha (lift, sell P into the pool, escrow E_S), then close by buying P back (escrow path) or by repaying P alpha from wallet and taking E_S.

Under the **buggy** design (readd via `adjust_protocol_liquidity`):

- **Sell-path / escrow close** reverses the forced sale then re-adds (τ_L, P). With an intervening pump the re-add still reweights at the skewed ratio (a15 drain).
- **Wallet-path close** returns $E_S + C$ for alpha repaid from outside, then re-adds (τ_L, P) with the same weight update — a **weight-ratchet** μ's fee-only F misses (see Section 6, Section 11).

Under the **fixed** design (Proposition 4), both closes credit reserves without touching w_1/w_2, so intervening swaps cannot crystallize into L.

5 TAO-pure cycles under the current design

Notation: attacker starts with TAO budget $V = h\,y$ (and, where needed, an alpha inventory acquired inside the cycle by a swap that is part of the cycle accounting).

5.1 Cycle A — long wallet ratchet (primary drain candidate)

1. (Optional) Sell alpha inventory into the pool (dump), moving spot down while the lending EMA lags.
2. Open long: post collateral C alpha, lift $g = P/y$, forced-buy escrow A with P TAO, subject to LTV, capacity, EMA band $p' \le \text{EMA}\,(1+\beta)$.
3. Close with `repay_from_wallet = true`: pay P TAO, receive $A + C$, re-add (P, α_L).
4. Sell returned escrow A for TAO.

Mark-to-market: all alpha legs netted; terminal asset is TAO. We write $F_A(h; y, x, w_1; \Theta)$ for V'/V.

Closed form (fee-free sketch, equal weights). Let $h = P/y \in (0,1)$, $w_1 = w_2 = \frac{1}{2}$, ignore swap fees and EMA. Lift removes (P, hx) from (y, x) in the sense $\alpha_L = hx$ when $g = P/y = h$. Forced buy of P on the lifted pool yields

$$A = (1-h)x\Big(1 - \sqrt{\frac{1-h}{1-h+h}}\Big) = (1-h)x\Big(1-\sqrt{1-h}\Big).$$

Post-open reserves $(y,\ x - \alpha_L - A) = (y,\ x(1-h)\sqrt{1-h})$. Wallet repay + weight update $\to$ reserves $(y+P,\ x-A)$ with spot locked at the post-open price; selling A then gives

$$F_A^{\text{update}}(h) = \frac{\Delta y_{\text{sell}}(A;\ y+P,\ x-A,\ w')}{P},$$

where w' is the reweighted pair. With *fixed* weights,

$$F_A^{\text{keep}}(h) = \frac{y+P}{P}\left(1 - \left(\frac{x-A}{x-A+A}\right)^{w_1/w_2}\right) = \frac{1+h}{h}\left(1-\left(1-\frac{A}{x}\right)\right) \quad (\text{for } w_1 = w_2)$$

reduces to a fee-free identity $F_A^{\text{keep}} < 1$ once the forced-buy A is substituted (exact algebra in increment 2). Numerics already show the sign split (Section 7).

5.2 Cycle B — short wallet mirror

Open short to the EMA band floor, wallet-repay with owned alpha, rebuy alpha (or equivalently value the free E_S against replacing P). Factor F_B.

5.3 Cycle C — raise then wallet-close

`open_long_and_sell` with $S = A$ (near price-neutral by design), then wallet-close the long and sell residual. Factor F_C.

5.4 Cycle D — escrow closes only

Open long or short, close on the escrow AMM path (no wallet repay). Fee-lossy by construction if re-add is the inverse of lift+trade. Factors $F_D^{\text{L}}, F_D^{\text{S}}$.

5.5 Cycle E — partial repay interleaved with swaps

Slice $k \in (0,1]$, swap, repay remainder. Factor F_E.

6 Where a fee-only proof fails

Proposition 1 (Weight update on asymmetric re-add)**.** *After a forced long buy the pool is at* $(y,\, x - \alpha_L - A)$ *with* ***unchanged*** *weights and elevated spot. Wallet repay credits* (P, α_L) *yielding reserves* $(y + P,\, x - A)$*. Calling* `update_weights_for_added_liquidity` *sets new weights so that the* elevated *spot is preserved. Then* $L = x^{w_1} y^{w_2}$ *(and* w_1*) move monotonically with repeated cycles — matching the observed "*w_1 *and* $\sqrt{k}$ *taking off".*

Consequently μ's F (fees only, fixed weights) is a **lower bound on pool protection** only for cycles that never hit the weight-updating wallet re-add. For Cycle A with weight update,

$$F_A \text{ can exceed } 1$$

even when the September F satisfies $F \geq 1$.

Remark 3 (Fix direction, not yet the proof). Re-adding wallet-repay legs *without* updating weights restores the market-buy residual $(+P, -A)$ under fixed (w_1, w_2). Numerics (Section 7) show that change flips Cycle A from $F_A > 1$ to $F_A < 1$ on SN29-like pools. A full proof that $F_A \leq 1$ under preserve-weights is deferred to a later increment.

7 Numerics: SN29-like and μ's F

Pool sketch: $y = 10^5$ TAO, $p \approx 0.00279$, $w_1 = w_2 = \frac{1}{2}$, $\varphi = 33/65535$. Side capacity $\sim 0.075\, y$–$0.165\, y$.

7.1 μ's F vs size

Evaluating (1)–(2) at $w_2 = \frac{1}{2}$, $\varphi = 33/65535$ (`scripts/no_arbitrage_F.py`):

f_h	F (fee only)	$F < 1$? (μ profit)
0.01	1.000485	no
0.05	1.000409	no
0.075	1.000365	no
0.10	1.000323	no
0.165	1.000224	no
0.25	1.000112	no

So on production fees, μ's fee-only F stays *just above* 1 for all listed sizes: the September fee argument alone would *rule out* attacker profit. The drain therefore cannot be explained by fees; it requires an extra mechanism (the weight ratchet).

7.2 Cycle A with weight update (counterexample)

Same pool ($y = 10^5$ TAO, $p \approx 0.00279$), $P = 0.165\, y$, multiplicative swap tax 0.5% per leg in the sketch model:

dump	weights	F_A	profit (τ)	w_1 after
0%	update	1.0176	+291	0.515
0%	keep	0.9631	−609	0.500
20%	update	1.0302	+498	0.522
20%	keep	0.9513	−804	0.500
35%	update	1.0413	+681	0.528
35%	keep	0.9411	−972	0.500
50%	update	1.0539	+889	0.535
50%	keep	0.9297	−1160	0.500

Counterexample: with weight-updating wallet re-add, $F_A > 1$ on every row; w_1 and L rise. With weight-preserving re-add, $F_A < 1$ on every row. JSON: `scripts/no_arbitrage_F.json`.

8 Theorem status (increment 1)

Conjecture 2 (No TAO-pure arb, preserve-weights wallet re-add)**.** *If wallet-repay re-adds lift legs without updating Balancer weights, and escrow closes re-add after reversing the forced trade, then for every TAO-pure cycle of length $\leq N$ (fees on, EMA fixed or free within band), $F_{\text{cyc}} \leq 1$, with equality only in the zero-fee, zero-size limit.*

Theorem 1 (Status)**.**

1. *Under **buggy** re-add (weight-updating via `adjust_protocol_liquidity`), Conjecture 2 is **false**: Cycle A and the clone short→pump→close sequences (Section 11) are counterexamples with $F_{\text{cyc}} > 1$ and durable w_1/L corruption.*
2. *μ's fee-only $F \geq 1$ does **not** imply $F_{\text{cyc}} \leq 1$ on that path, because F omits the weight ratchet.*
3. *Under design (a) (Proposition 4 / Theorem 3): bounded cycle search finds no $F_{\text{cyc}} > 1$; clone regressions assert no attacker profit and unchanged w_1. Full closed-form proof of Conjecture 2 remains **open**.*

9 h-sweep and μ's returns graph

9.1 μ's graph (September), eyeball check

Image: return (%) vs pool size in units of the trader's TAO budget (log x, 10%–1000%). Eyeball (verified against the figure):

- $\approx +1\%$ at 10% (pool $= 0.1\times$ budget $\Rightarrow h =$ budget/pool $= 10$ in that proposal's units);
- peak $\approx +4.5\%$ near 100%;
- crosses 0 near $\sim 300\%$;
- $\approx -39\%$ at 1000%.

So that sequence is profitable when the budget is at least about a third of pool TAO — the **large-trade** regime, where lift/AMM curvature dominates fees.

9.2 Cycle A under current lift (production fee)

Sweep $h = P/y$ on an SN29-like pool ($y = 10^5\,\tau$, $p \approx 0.00279$, $\varphi = 33/65535$), weight-update vs preserve-weights (`scripts/no_arbitrage_h_sweep.json`):

h	pool/budget	ret% update	ret% keep	w_1 (upd)
0.01	10000%	−0.09	−0.11	≈ 0.500
0.03	3333%	−0.01	−0.19	≈ 0.500
0.05	2000%	+0.15	−0.35	0.501
0.072	1381%	+0.43	−0.62	0.503
0.165	606%	+2.71	−2.81	0.515
0.30	333%	+9.75	−9.08	0.557
0.50	200%	+31.1	−25.1	0.666

Breakeven for the update path: $h_\star \approx 0.031$ (below: fee-dominated loss; above: ratchet profit). Qualitative agreement with μ: **large h is where EV flips positive**. The quantitative September curve is from the earlier proposal (different fee placement / no forced-escrow lift), so magnitudes differ; both point at the same regime.

10 Do existing protections keep $EV \leq 0$?

Reachable h on a **single** open (equal weights):

- EMA band +15%: fee-free buy move $p'/p = (1+h)^{1/w_1} = (1+h)^2 \leq 1.15 \Rightarrow h \leq \sqrt{1.15} - 1 \approx 0.0724$.
- Side capacity: ≈ 0.075 (thin) to ≈ 0.165 (floor × split).
- `PrincipalExceedsTaoIn` / `AlphaIn`: $P < y$ (and symmetrically for shorts) — far looser than the band/capacity.

At the EMA cap ($h \approx 0.072$) with weight-update Cycle A: $F_A - 1 \approx +0.43\%$ — **still positive EV**. At capacity $h = 0.165$ (only reachable if spot was depressed or the EMA lagged): $\approx +2.7\%$. Preserve-weights is negative at both points.

Splits and multiple coldkeys. Same-side opens are gated by the *sticky* EMA: after one long, spot is up and further longs hit the band sooner. That slows naive splitting, but does not zero EV:

- N sequential opens at $h = 0.05$ (inside the band) each earn $\approx +0.15\%$ while w_1 ratchets ($0.500 \to 0.513$ by $N = 10$).
- A pre-dump that pushes spot below the EMA, then one long up to EMA · 1.15, realises a *much* larger effective h and the 10^2–$10^3\,\tau$ profits of Section 7.
- Multiple coldkeys in one block still serialize; they share the same post-trade spot/EMA constraint, but can alternate sides or wait for EMA catch-up.

Theorem 2 (Protections vs Cycle A)**.** *EMA band, side capacity, and principal-vs-reserve checks **bound** h but **do not** imply $F_A \leq 1$ on the weight-updating wallet path. Inside the reachable set, $F_A > 1$ for all $h \gtrsim 0.031$. Only weight-preserving wallet re-add flips the sign through the whole sweep.*

11 Clone reproduction (spec 493)

On a live clone tip `4d3dc1585` (shallow SN29-like book, $y \approx 7.5 \cdot 10^3\,\tau$, $p \approx 0.00265$, $w_1 \approx 0.47$):

seq	path	attacker Δ	SubnetTAO Δ	w_1 jump
a13	short 1500 → foreign pump 9000 → wallet repay	$+289\,\tau$	$-298\,\tau$	$0.4717 \to 0.4506$
a15	short 1200 → self-pump 7000 → buyback	$+122\,\tau$	$-128\,\tau$	$0.4507 \to 0.4389$
a14	wallet repay, *no* pump (control)	$-0.61\,\tau$	$+0.11\,\tau$	—

On profitable closes, $L \approx x^{1-w_1} y^{w_1}$ jumps with w_1 (a13: 174k → 203k) and **does not revert** after unstake. AlphaIn mean-reverts; the durable corruption is w_1/L, which lets the exit unstake extract more TAO than the pump deposited.

Mechanism.

1. `lift_protocol_liquidity` removes $(g\,y, g\,x)$ **without** updating weights (proportional remove).
2. Intervening `AddStake`/swap skews the reserve ratio; weights stay put.
3. Buggy `readd_protocol_liquidity` routed through `adjust_protocol_liquidity` → `update_weights_for_` at the **post-pump** ratio, rewriting w_1/w_2 and inflating L.
4. Subsequent `RemoveStake` against corrupted weights is extractive.

This is μ's open→swap→close framing with $F_{\text{cyc}} > 1$ driven by the weight ratchet, not by fee-only $F < 1$.

12 Answer to mu: we do not abandon Balancer

Remark 4 (Scope of the bypass). Ordinary **stake / unstake** (and every other AMM swap) still execute on the Balancer weighted pool with the stored (w_1, w_2):

$$p = \frac{w_1}{w_2}\frac{y}{x}, \qquad L = x^{w_1} y^{w_2}.$$

Emissions still call `adjust_protocol_liquidity`, which may `update_weights_for_added_liquidity` so a one-sided injection keeps spot. **Only** lending's `lift`/`readd` pair bypasses that weight update. The bypass is not "leave Balancer" — it is "use the proportional remove/add inverse, not the asymmetric-join reweight."

Proposition 2 (Coherence while a position is open)**.** *After a lift of fraction g, the live pool is $((1-g)y, (1-g)x, w_1, w_2)$. Spot is unchanged, $L \mapsto (1-g)L$, and every subsequent stake/unstake is a standard Balancer swap on that thinner book. Custody holds the removed legs; they are not part of the AMM state. So pricing during the open interval is exactly Balancer.*

13 Two readd designs

Lift always snapshots the removed legs $(\Delta y, \Delta x) = (gy, gx)$ and the invariant slice $\Delta L = g\,L$. After intervening swaps the live ratio may differ. Two ways to close the Balancer side:

(a) Snapshot / restore exact reserve deltas (weights untouched). Credit exactly $(\Delta y, \Delta x)$ into current reserves; leave (w_1, w_2) alone.

- No intervening swaps: exact inverse — p and L restored.
- With intervening swaps: w unchanged; L after credit is *not* generally L_0 (homogeneous add of a stale-ratio vector into a skewed book). Spot may jump on the credit.
- This is an unbalanced join *without* the Balancer price-preserving weight rewrite (and without an extra join fee).

(b) Settle in invariant units (L-conserving proportional join). Treat the position as owning ΔL LP shares. On readd, join proportionally at the *current* ratio, $(sy_{\text{skew}}, sx_{\text{skew}})$ with $s = \Delta L / L_{\text{skew}}$, so spot is preserved and L rises by ΔL. Custody holds $(\Delta y, \Delta x) \neq (sy, sx)$; the residual must be mark-to-market (one Balancer swap). That residual swap is a new surface.

Proposition 3 (Buggy readd is neither (a) nor (b))**.** *`adjust_protocol_liquidity`→`update_weights_for_added_` credits $(\Delta y, \Delta x)$ and solves for new weights that keep post-credit spot equal to the skewed mark. That is the asymmetric-join tool for emissions, not the inverse of a proportional lift. Empirically it inflates L and unlocks extractive unstake (a13/a15).*

14 Root fix = design (a)

Proposition 4 (Design (a) is the Balancer inverse of lift)**.** *Lift of fraction g yields $((1-g)y, (1-g)x, w_1, w_2)$. Crediting the same legs $(\Delta y, \Delta x)$* ***without*** *changing weights is the set-theoretic inverse on the no-swap path. With intervening swaps, (a) still leaves (w_1, w_2) invariant (the property stake/unstake need); it does* not *claim $L = L_0$ after credit.*

Corollary 1 (Implementation)**.** *`readd_protocol_liquidity` = pure reserve credit (no weight write, no reservoir). Emissions keep calling `adjust_protocol_liquidity`. Lending `readd_and_credit` / `_preserve_weights` both use (a).*

15 Bounded cycle search

Script: `scripts/no_arbitrage_cycle_search.py`. Enumerates short/long × buyback/wallet-close × pump/dump sizes on SN29-like and expanded (w_1, y) grids. Stake/unstake always Balancer at fixed w; only the readd design varies.

design	cells	profitable ($F_{\text{cyc}} > 1$)	max F_{cyc}
buggy (weight update)	672	318	1.88
(a) exact deltas, keep w	672	0	0.999
(b) ΔL join + residual MTM	672	0	0.999

a13/a15-like points: buggy $F > 1$ with w_1/L jump; (a) and (b) both $F < 1$ with w_1 fixed ((b) also keeps L-ratio = 1).

Theorem 3 (Status after search)**.** *1. We do* ***not*** *abandon Balancer: swaps stay weighted; only lending readd skips `update_weights_for_added_liquidity`.*

2. *Design (a) is coherent while a position is open (Proposition 2).*

3. *On the enumerated TAO-pure cycles, (a) and (b) both have* $\max F_{\text{cyc}} < 1$*; buggy does not.*

4. *This is a* ***bounded search****, not a closed-form proof over all cycles / continuous sizes / multi-block EMA games. A full proof of Conjecture 2 remains open. We ship (a) because it matches the lift inverse, kills the reproduced drain, and clears the search; (b) also clears the search but needs residual-swap accounting we are not adding in this fix.*

Regression coverage (pallet tests):

- `test_readd_after_skew_preserves_weights` (swap);
- `sn29_short_self_pump_buyback_close_...` (a15);
- `sn29_short_foreign_pump_wallet_repay_...` (a13).

16 Follow-up: "is this just Uniswap v2?"

μ: *if you don't change (or re-derive) the weights, it's basically back to Uniswap v2.*

16.1 What main actually re-derives

Main's Balancer has **two** distinct operations:

1. **Swaps** (stake / unstake): reserves move on the curve $L = x^{w_1} y^{w_2}$ at the *stored* weights. Weights are **not** re-derived mid-swap. This was already true before lending existed.

2. **Asymmetric liquidity adds** (emissions via `adjust_protocol_liquidity`): reserves receive an arbitrary $(\mathrm{d}y, \mathrm{d}x)$; then `update_weights_for_added_liquidity` solves for new (w_1, w_2) so that post-add spot equals pre-add spot. That is the price-neutral join. It is *not* part of the swap path.

So "Balancer behaviour" on main = fixed-weight weighted CPMM for trades, plus optional weight re-derivation on protocol/user *liquidity adds* so those adds do not move price.

16.2 Fixed weights $\neq$ Uniswap v2

Uniswap v2 is the **special case** $w_1 = w_2 = \frac{1}{2}$ of the Balancer invariant ($L = \sqrt{xy}$, $p = y/x$). With $w_1 \neq w_2$ held fixed, the pool is still a **weighted** Balancer CPMM:

$$p = \frac{w_1}{w_2} \frac{y}{x}, \qquad L = x^{w_1} y^{w_2},$$

with the same swap closed forms main already uses. SN29-like books run near $w_1 \approx 0.47$, not $\frac{1}{2}$. Design (a) does not force equal weights; it refuses to *rewrite* them on lending readd.

16.3 What is true while a lift is outstanding

After lift, (w_1, w_2) are unchanged (lift is proportional). Every swap against the thinner book is therefore a fixed-weight Balancer swap — **exactly as every swap already was**. Lending did not "switch the AMM off Balancer"; swaps never consulted the asymmetric-join reweight. The only behavioural change under (a) is on the **readd after skew**: instead of a price-neutral asymmetric join (reweight to keep the post-pump spot), the protocol credits the snapshot legs at fixed w, which can move spot and need not restore L_0.

16.4 What is lost versus "full" Balancer join behaviour?

Surface	Under design (a)
Stake / unstake	Unchanged: weighted CPMM at stored w.
Emissions / protocol adds	Unchanged: still `adjust_protocol_liquidity` + weight re-derivation.
User LP-style asymmetric adds	Unchanged (lending is not that path).
Lending readd after skew	**Lost:** price-neutrality of the asymmetric join. Readd may move spot; L may drift from L_0.

Is that acceptable? Yes, as an emergency root fix: readd is the protocol restoring its own previously removed inventory, not a third-party LP joining at the mark. The property we must not give attackers is "crystallize a pumped spot into w and inflate L." Giving up price-neutral readd for that one path is the cost. Bounded search (Section 15): no profitable cycle on the grid. Full proof still open.

17 Designs that keep re-derivation elsewhere

Goal: emissions / other asymmetric adds still re-derive weights; lending's open→swap→close cycle is non-extractable.

(a) — shipped. Lending: fixed-w credit of snapshot deltas. Elsewhere: re-derive as today. Pros: smallest diff; exact lift inverse on the quiet path; kills a13/a15; search-clean. Cons: readd can move spot; L not restored after skew; proof open.

(b) — ΔL proportional join + residual MTM. Lending readd never needs weight re-derivation (join is proportional at the *current* ratio). Residual custody vs proportional basket is one Balancer swap (fees apply). Pros: spot preserved on join; L restored by construction; search-clean; closest to "Balancer LP semantics." Cons: new residual-swap accounting; residual is a surface to audit; more code.

(c) — re-derive against the *lift-time* ratio. Idea: call `update_weights_for_added_liquidity` with the lift-time book (or the lift ratio) instead of the skewed mark. But the lift legs $(\Delta y, \Delta x)$ are *proportional* to the lift-time reserves, so a weight update computed against that ratio is a **no-op** — same w as (a). If instead one reweights the *skewed* post-credit reserves so that spot equals the **lift-time** spot, that forces price back to a historical level and is a different (likely more dangerous) policy. Verdict: "re-derive at lift-time ratio" for proportional lift legs collapses to (a); the force-price-to-lift-spot reading is not recommended.

(d) — charge / target the EMA ratio on readd. Variant: proportional join aimed at the lending EMA price, or reweight so post-readd spot equals EMA. Pros: ties exit to the same oracle the band/settlement already use. Cons: EMA can lag (the band exists because of that); targeting EMA on readd can itself be gamed by pre-moving spot then waiting; does not by itself explain a13 (drain was weight crystallisation at the *current* mark, not EMA). Useful as a settlement policy later; not the minimal drain fix.

(e) — keep current-mark reweight but haircut / fee the join. Still rewrites w toward the pumped mark; a fee may shrink but not remove the ratchet under repetition. Rejected as the primary fix.

18 Recommendation for Jacob

1. **Ship design (a) now** (already in the swap pallet): lending lift/readd keep weights fixed; emissions keep re-deriving. This does *not* reduce the AMM to Uniswap v2 unless weights were already $\frac{1}{2}$; swaps remain weighted Balancer. What is lost is only price-neutral lending readd after skew — acceptable against the reproduced drain.

2. **Do not** restore current-mark `update_weights_for_added_liquidity` on lending readd. That is the bug.

3. **Next hardening (optional, preferred if (a) ever shows a counterexample):** design (b) — restore ΔL by a current-ratio proportional join and mark-to-market the custody residual with an explicit Balancer swap (fee on). Keeps re-derivation for emissions; gives price-neutral lending readd without crystallising the pump into w.

4. Treat (c) "lift-time re-derive" as equivalent to (a) for proportional legs; do not chase (d)/(e) as the root fix.

5. Status: bounded cycle search clears (a) and (b); closed-form no-arb proof remains open. Skeptic/Auditor should treat (a) as "search-backed emergency inverse," not "theorem-closed."

19 Next increments

- Closed-form proof (or counterexample) for (a) beyond the grid.
- Spec + implementation sketch for (b) residual MTM if needed.
- Call-flow diagram (read state once / validate / derive / act).

References

[1] μ, September 2025 hand derivation of fee factor F (image on PR discussion). Profitability under the earlier proposal when $F < 1$.

[2] *Lift-based lending on main's Balancer pools*, v5.4, `pallets/lending/docs/lift-lending-spec.md`.